Faculty of Engineering

Department of Informatics Engineering

Software Engineering and Information Systems

Artificial Intelligence Engineering and Data Science

Drug management platform

A junior project report - submitted to complete the requirements for obtaining a bachelor's degree in informatics engineering -Artificial Intelligence Engineering and Data Science

Prepared by

Ali Ahmed Al-Obeid

Mohammed Ibrahim Al-Saeed

Abdulkarim Salaymeh

Supervised by

Eng. Maher Sarem

كليـــــــــــة الهندســـــــــــــة
قســـــم الهندســـــة المعلـوماتيـــة
هندسة البرمجيات ونظم المعلومات
هندسة الذكاء الصنعي وعلوم البيانات

منصة ادارة الادوية

تقرير للمشروع الفصلي قدّم استكمالاً لمتطلبات الحصول على درجة

البكالوريوس في هندسة المعلوماتية - هندسة الذكاء الصنعي وعلوم البيانات

إعداد

علي أحمد العبيد

محمد إبراهيم السعيد

عبدالكريم سلايمة

اشراف

المهندس. ماهر صارم

Supervisor Certification

I certify that the preparation of the junior project report entitled "Medication Management Platform" was carried out by Ali Ahmad Al-Obeid, Mohammed Ibrahim Al-Saeed, and Abdulkarim Salaymeh under my supervision at the Faculty of Engineering, Department of Informatics Engineering, in partial fulfillment of the requirements for the Bachelor Degree in Informatics Engineering.

Project title: Medication Management Platform

Supervisor: Eng. Maher Sarem

Name: .....

Signature: .....

Date: .....

Dedication

We dedicate this work to our families, teachers, and colleagues whose support encouraged us to complete this project. We also dedicate it to every patient who needs a simple digital tool to organize medication schedules and avoid missed doses.

Acknowledgment

We express our sincere appreciation to Eng. Maher Sarem for guidance and feedback during the analysis, design, implementation, and documentation stages. We also thank the faculty members who provided the academic standards used to improve the structure and quality of this report.

Arabic Abstract

يقدم هذا المشروع منصة ويب لإدارة الأدوية تساعد المريض على تنظيم أدويته وجدولة الجرعات واستقبال التنبيهات المرتبطة بكل جرعة. تعالج المنصة مشكلة شائعة في الرعاية الصحية وهي ضعف الالتزام الدوائي الناتج عن النسيان أو تعدد الأدوية أو عدم الانتباه إلى قرب نفاد الكمية. تتكون المنصة من واجهة للمريض لإضافة الدواء من كتالوج النظام وتحديد الكمية وحد التنبيه وجدولة الجرعات ضمن فترات واضحة، وواجهة للمدير لإدارة المرضى وكتالوج الأدوية مترابطة بمفاتيح أساسية وخارجية، وعلى منطق MySQL يعتمد النظام على قاعدة بيانات. وإرسال التنبيهات التحذيرية خلفي يتحقق من مواعيد الجرعات والكمية المتبقية ويمنع تكرار الإشعارات. تم توثيق المشروع وفق منهجية هندسة البرمجيات من خلال تحليل المتطلبات، تحديد أصحاب المصلحة، نمذجة حالات الاستخدام، التصميم المعماري، تصميم قاعدة البيانات، خطة الاختبار، ونتائج التقييم. يعد المشروع أساسا قابلا للتوسع نحو تحليلات الالتزام الدوائي والتنبؤ بخطر تفويت الجرعات في الإصدارات المستقبلية.

الكلمات المفتاحية: إدارة الأدوية، الالتزام الدوائي، تنبيهات الجرعات، MySQL، هندسة البرمجيات، واجهة ويب،.

Abstract

Medication adherence is a practical healthcare problem that becomes harder when patients use multiple medications, follow different schedules, or cannot easily notice that a medication quantity is close to running out. The Medication Management Platform proposed in this report is a web-based system designed to help patients organize medications, schedule daily doses, receive contextual reminders, and manage low-stock alerts in one place. The platform provides two primary workspaces. The patient workspace allows users to create an account, search the medication catalog, add medications to a personal list, define a dose schedule using clear morning and evening periods, confirm taken or missed doses, and view notifications. The administrator workspace supports patient account management, medication catalog management, and manual warning notifications. The system is designed as a layered web application with a Python backend, a browser-based user interface, a MySQL relational database, role-based access control, JSON Web Token authentication, and background scheduling logic for reminders and low-stock checks. The report follows a scientific software engineering structure: project introduction, development methodology, literature and existing-system review, requirements analysis, UML modeling, architecture and database design, implementation environment, testing, results, and recommendations. The implemented intelligence is rule-based rather than machine-learning-based; it focuses on automated reminders, duplicate-notification prevention, and stock-threshold monitoring. Future extensions can add adherence analytics, predictive risk scoring, integration with pharmacy data, and mobile push notifications. The final outcome is a maintainable case study in health software engineering that can support patients and administrators while remaining open for future data analytics and intelligent healthcare features.

Keywords: medication management, medication adherence, dose reminders, low-stock alert, web platform, MySQL, role-based access control, software engineering.

Table of Contents

Table P-1: Static table of contents

Section	Description	Page
Preliminary Pages	Dedication, certification, abstracts, lists, and glossary	
Chapter 1: General Introduction	Project overview, problem statement, goals, scope, contribution, and document structure	
Chapter 2: Development and Project Management Methodology	Agile approach, timeline, risk management, team roles, quality, and SCM	
Chapter 3: Literature Review and Existing Systems	Related systems, comparative analysis, research gap, and future trends	
Chapter 4: Requirements Modeling and Analysis	Feasibility, stakeholders, functional and non-functional requirements, UML, and traceability	
Chapter 5: System Architecture and Design	High-level architecture, database design, APIs, and security	
Chapter 6: Implementation Environment and Tools	Languages, frameworks, development environment, database server, and dependencies	
Chapter 7: Software Implementation	Project structure, modules, APIs, data access, business logic, UI, and smart logic	
Chapter 8: Testing and Evaluation	Test plan, test types, cases, results, metrics, and usability	
Chapter 9: Results and Analysis	System outputs, objective fulfillment, strengths, weaknesses, and KPIs	
Chapter 10: Conclusion and Recommendations	Summary, challenges, future work, and recommendations	
Appendices and References	Supplementary material, user manual, and references	

List of Figures

Table P-2: List of figures

List of Tables

Table P-3: List of tables

Glossary of Terms

Table P-4: Glossary

Term	Definition
Medication adherence	The degree to which a patient takes medications according to the prescribed plan.
Dose schedule	A structured plan that defines the days and time slots in which a dose should be taken.
Low-stock alert	A notification generated when the remaining medication quantity reaches a threshold set by the patient.
RBAC	Role-Based Access Control; a security model that grants permissions according to user roles such as Patient and Admin.
JWT	JSON Web Token; a token format used for stateless authentication.
REST API	A web API style that exposes resources through HTTP methods such as GET, POST, PUT, and DELETE.
Soft delete	A deletion method that marks a record as inactive instead of physically removing it from the database.
Scrum	An Agile framework that organizes work into short iterations called sprints.
SCM	Software Configuration Management; processes and tools used to control versions and changes.
KPI	Key Performance Indicator; a measurable value used to evaluate project or system performance.

Chapter 1: General Introduction

1.1 Project Overview and Background

Digital health systems increasingly support care outside the traditional clinical visit. Patients often need continuous assistance in remembering medication doses, distinguishing between different medications, and noticing when a medication quantity is close to depletion. The proposed Medication Management Platform, also referred to as MedTrack, is a web application that organizes these activities into one structured environment.

The project combines a patient interface with an administrator interface. The patient can search the medication catalog, add a medication to a personal medication list, define schedule information, receive notifications, and confirm dose status. The administrator can manage the general medication catalog, view patient data, and send warning notifications. The platform therefore connects medication records, schedules, reminders, and administrative control in a single system.

1.2 Problem Statement and Significance

The practical problem addressed by the project is the lack of a simple and integrated mechanism that helps patients organize medications and respond to reminders from one place. Generic alarms can notify a user at a time, but they are usually disconnected from medication ownership, dose history, low-stock state, and administrative follow-up.

From a practical perspective, the platform reduces forgetfulness, improves the clarity of the medication plan, and gives the administrator a consistent way to manage patient and medication data. From an academic perspective, the project is a case study in health-oriented software engineering. It includes requirements analysis, relational database design, role-based protection, background scheduling, user interface design, and testing.

1.3 Project Objectives

- Build a web platform that enables patients to manage personal medications and daily dose schedules.
- Provide catalog search by medication name and category, such as cardiac, psychiatric, diabetes, and analgesic medications.
- Generate automatic dose reminders based on the actual dose schedule instead of independent manual alarms.
- Link each confirmation action to the correct notification and dose record.
- Create low-stock alerts when the remaining quantity reaches a threshold defined by the patient.
- Provide an administrator dashboard for patient management, medication catalog management, and warning notifications.
- Use a relational MySQL database with primary keys, foreign keys, and constraints that protect data consistency.

- Maintain clear request and response behavior with understandable error messages and validation rules.

1.4 Scope, Constraints, and Assumptions

- Scope: The project covers a web application, not a native mobile application. It includes patient and administrator roles, medication catalog functions, dose schedules, notifications, and low-stock alerts.
- Constraints: Development time is limited to a junior project period, real hospital integration is not available, and clinical prescription validation is outside the scope.
- Assumptions: Users have basic ability to use a web browser, administrators are authorized by the institution, and medication information is entered accurately in the catalog.
- Boundary: The system is a medication organization and reminder tool. It does not replace medical diagnosis, prescription decisions, or pharmacist counseling.

1.5 Contribution to the Field

The project contributes a focused software engineering model for medication organization. Its main contribution is not a new medical algorithm, but a practical integration of medication catalog management, patient medication ownership, dose scheduling, contextual notifications, low-stock checks, and administrator follow-up. This integrated design can later be extended with adherence analytics and predictive risk models.

1.6 Research Methodology and Scientific Approach

The report follows an engineering research methodology. The problem is identified, requirements are collected and modeled, the system is designed using UML and database modeling, implementation choices are documented, and testing is used to evaluate whether the system meets its requirements. The software development process itself follows an Agile/Scrum approach, which is detailed in Chapter 2.

1.7 Document Structure

Chapter 2 explains development and project management methodology. Chapter 3 reviews related systems and identifies the project gap. Chapter 4 models and analyzes requirements. Chapter 5 presents architecture and design. Chapters 6 and 7 document implementation environment and code-level organization. Chapter 8 presents testing and evaluation. Chapters 9 and 10 discuss results, conclusions, and recommendations. Appendices contain supporting material and user guidance.

1.8 Basic Definitions

The most important terms used in the report are defined in the glossary. In this document, the term medication means a catalog item that can be added by the patient, while patient medication means the patient-specific record that stores quantity, activity state, and low-stock

threshold. A dose record is a logged occurrence of a scheduled dose and can be marked taken or missed.

Chapter 2: Development and Project Management Methodology

2.1 Selected Development Methodology and Justification

The project adopts Agile development using Scrum principles. This choice is appropriate because the user interface, notification logic, and administrator operations can be improved incrementally. Each sprint implements and tests a small part of the system, allowing feedback and corrections before adding more features.

2.2 Project Schedule

Table 2-1: Project schedule

Week	Phase	Main Tasks	Deliverable
1	Requirements analysis	Define problem, actors, core features, constraints, and acceptance criteria	Requirement list and initial backlog
2	Architecture and database design	Prepare high-level architecture, ERD, and relational schema	Architecture diagram and database model
3	Backend APIs	Implement authentication, medication catalog, patient medication APIs	Working backend endpoints
4	Patient interface	Build medication search, add medication, schedule, notifications	Patient dashboard
5	Admin dashboard	Build patient management, catalog management, warning notifications	Admin screens
6	Scheduling and alerts	Implement dose reminders, low-stock checks, duplicate prevention	Notification logic
7	Testing and fixes	Run manual and smoke tests, fix defects	Test report
8	Documentation and delivery	Prepare final report, diagrams, and user manual	Final report and project package

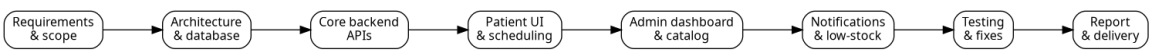

Figure 2-1: Project timeline and main development phases

2.3 Risk Management

Table 2-2: Risk register and treatment strategy

ID	Risk	Probability	Impact	Mitigation
R1	Unclear requirements	Medium	High	Use short sprint reviews and maintain a prioritized backlog.
R2	Database inconsistency	Medium	High	Use primary keys, foreign keys, unique constraints, and validation.
R3	Duplicate notifications	High	Medium	Check existing unread reminders before creating a new notification.
R4	Authentication weakness	Medium	High	Use JWT expiration, role guards, and token revocation/blocklist.
R5	Limited testing time	Medium	Medium	Define smoke tests for the full patient and admin journeys early.
R6	Interface complexity	Low	Medium	Use simple screens, clear labels, and consistent error messages.

2.4 Team Roles and Task Distribution

Table 2-3: Team roles and responsibilities

Role	Responsibility Area	Tasks
Team member 1	Requirements analyst / documentation lead	Collect requirements, maintain report structure, prepare diagrams.
Team member 2	Backend developer	Implement authentication, APIs, database access, and scheduling logic.
Team member 3	Frontend developer / tester	Build user interfaces, run manual tests, and record defects.
Supervisor	Academic reviewer	Review scope, methodology, documentation quality, and final presentation.

2.5 Quality Management

Quality is managed through consistent validation, database constraints, role-based access control, a simple user interface, and smoke tests that cover major journeys. Acceptance criteria are written for each main requirement. The system is considered acceptable when the

patient and administrator journeys can be completed without inconsistent data or unauthorized access.

2.6 Project Management Tools

The team can manage the backlog using a spreadsheet, Trello, Notion, or GitHub Projects. Code and documentation changes should be stored in Git to ensure traceability and rollback. Visual Studio Code is used as the primary development editor.

2.7 Monitoring and Continuous Evaluation

Progress is monitored through weekly reviews. Each review checks completed backlog items, remaining defects, changes in scope, and documentation status. The most important evaluation points are successful authentication, correct scheduling, correct low-stock alerts, and successful administrator actions.

2.8 Software Configuration Management

The recommended SCM strategy uses a main branch for stable versions, a dev branch for integration, and feature branches for individual changes. Each feature branch should be reviewed before merging. Version tags can be used for presentation builds, such as v1.0-demo and v1.0-final.

Chapter 3: Literature Review and Existing Systems

3.1 Related Systems and Previous Work

Medication management tools appear in several forms: pharmacy delivery services, mobile reminder applications, and general digital health platforms. The original project report discussed PillPack and Vezeeta as reference systems. This revised report uses them as examples and adds a comparison model that focuses on the gap addressed by MedTrack.

3.2 Modern Technologies in the Field

Modern web health systems commonly use browser-based frontends, REST APIs, relational databases, role-based security, background jobs, and notification services. In more advanced systems, data analytics and machine learning can be used to detect adherence patterns, identify patients at risk of missing doses, and recommend interventions. MedTrack currently implements rule-based smart behavior and leaves predictive analytics for future work.

3.3 Comparative Analysis of Existing Systems

Table 3-1: Comparative analysis of existing and proposed systems

System	Main Features	Strengths	Limitations
PillPack / pharmacy delivery model	Medication packaging, refill coordination, delivery tracking, reminders	Strong pharmacy integration and refill support	Depends on pharmacy logistics and may not focus on local academic deployment
Vezeeta pharmacy services	Search medicines, locate pharmacies, upload prescriptions, compare availability	Strong marketplace and service ecosystem	Reminder and patient-owned dose history are not the main focus
Generic reminder apps	Manual alarms and repeated reminders	Simple and fast to configure	Weak connection to medication stock, dose records, and administrator management
MedTrack proposed platform	Patient medication list, schedules, contextual reminders, low-stock alerts, admin dashboard	Integrated data model and role-based management for a project setting	No real pharmacy integration or mobile push notifications in the current version

3.4 Research Gap and Proposed Innovation

The identified gap is the absence of a simple integrated project platform that links a patient-specific medication record to dose scheduling, notification creation, dose confirmation, remaining quantity, and administrator follow-up. MedTrack addresses this gap through a

unified data model and background checks that create reminders and low-stock notifications automatically.

3.5 Theoretical and Reference Framework

The system is based on REST architecture for APIs, a layered software architecture for separation of concerns, relational database theory for data consistency, and RBAC for security. The scheduling logic is deterministic: it calculates whether a dose is due and whether the remaining quantity is below the configured threshold.

3.6 Modern and Future Trends

Future digital health systems can benefit from mobile push notifications, wearable-device integration, data analytics dashboards, AI-based adherence risk prediction, and secure interoperability with pharmacy or hospital systems. These trends are outside the immediate junior project scope, but they guide the future roadmap.

Chapter 4: Requirements Modeling and Analysis

4.1 Feasibility Study

The project is technically feasible because it uses common web technologies: Python for backend services, JavaScript for the browser interface, and MySQL for data storage. It is financially feasible for an academic environment because it can run locally without paid cloud services. The main feasibility limitation is the lack of real clinical integration and official medicine databases.

4.2 Requirement Elicitation

Requirements were derived from the original project scope, review of similar systems, and analysis of the patient and administrator journeys. The core user need is to receive accurate reminders that are connected to specific medications and records. The core administrator need is to manage patient data, catalog data, and warnings in a controlled interface.

4.3 Stakeholders

Table 4-1: Stakeholder analysis

Stakeholder	Role	Needs and Expectations
Patient	Uses the platform to organize medications and confirm doses	Simple interface, accurate reminders, clear notifications, privacy
System Administrator	Manages catalog, patients, and warnings	Role-protected dashboard, search, edit, disable, and notification tools
Project Team	Develops and maintains the system	Clear requirements, testability, modular code, database consistency
Supervisor / Evaluator	Reviews academic quality and project correctness	Scientific structure, traceability, testing evidence, maintainability
Future Pharmacist or Healthcare Provider	Possible future integration user	Reliable medication data, communication mechanisms, reporting

4.4 Functional Requirements

Table 4-2: Functional requirements

ID	Requirement
FR-01	The patient shall be able to create an account and log in securely.
FR-02	The patient shall be able to search and browse the medication catalog by name or category.
FR-03	The patient shall be able to add a medication to a personal medication list

ID	Requirement
	with quantity and low-stock threshold.
FR-04	The patient shall be able to define and edit dose schedules using clear time slots such as morning and evening.
FR-05	The system shall create a contextual reminder when a scheduled dose is due.
FR-06	The patient shall be able to mark the dose as taken or missed from the related notification context.
FR-07	The system shall reduce or update remaining quantity when a dose is confirmed according to the business rule.
FR-08	The system shall create a low-stock alert when remaining quantity reaches the configured threshold.
FR-09	The administrator shall be able to view, search, edit, and disable patient accounts.
FR-10	The administrator shall be able to add and update medications in the system catalog.
FR-11	The administrator shall be able to send warning notifications to selected patients or groups.
FR-12	The system shall allow users to view and mark notifications as read.

4.5 Non-Functional Requirements

Table 4-3: Non-functional requirements

Category	Requirement	Achievement Mechanism
Performance	Core operations should respond in less than three seconds in a local test environment.	Pagination, indexes, and efficient queries.
Security	Routes and records must be protected by role and ownership.	JWT, role_required decorators, guards, and token blocklist.
Usability	The interface should be clear and not crowded.	Consistent buttons, simple labels, and clear error messages.
Maintainability	Code should be modular and easy to extend.	Separate routes, services, repositories, validators, and models.
Data integrity	The database must prevent duplicates and protect relationships.	Unique constraints, foreign keys, soft delete, and validation.
Testability	The patient and administrator journeys must be testable.	Smoke test scripts and manual test cases.
Scalability	The architecture should support	REST API, layered services, and

Category	Requirement	Achievement Mechanism
	future analytics and mobile clients.	normalized data model.

4.6 Requirement Prioritization

Table 4-4: MoSCoW prioritization

Priority	Requirements
Must	Authentication, medication catalog, personal medication list, dose schedule, dose reminder, low-stock alert, admin login
Should	Notification read status, patient search, catalog filtering, clear validation errors
Could	Adherence reports, exportable logs, dark mode, multilingual UI
Won't for current version	Real pharmacy integration, electronic prescriptions, clinical diagnosis, payment, insurance processing

4.7 UML Modeling

The guide recommends using a limited number of main use cases and focusing on the operations that represent the core value of the system. Therefore, this revised report presents the main use case diagram and two representative activity diagrams instead of listing every small screen action as a separate diagram.

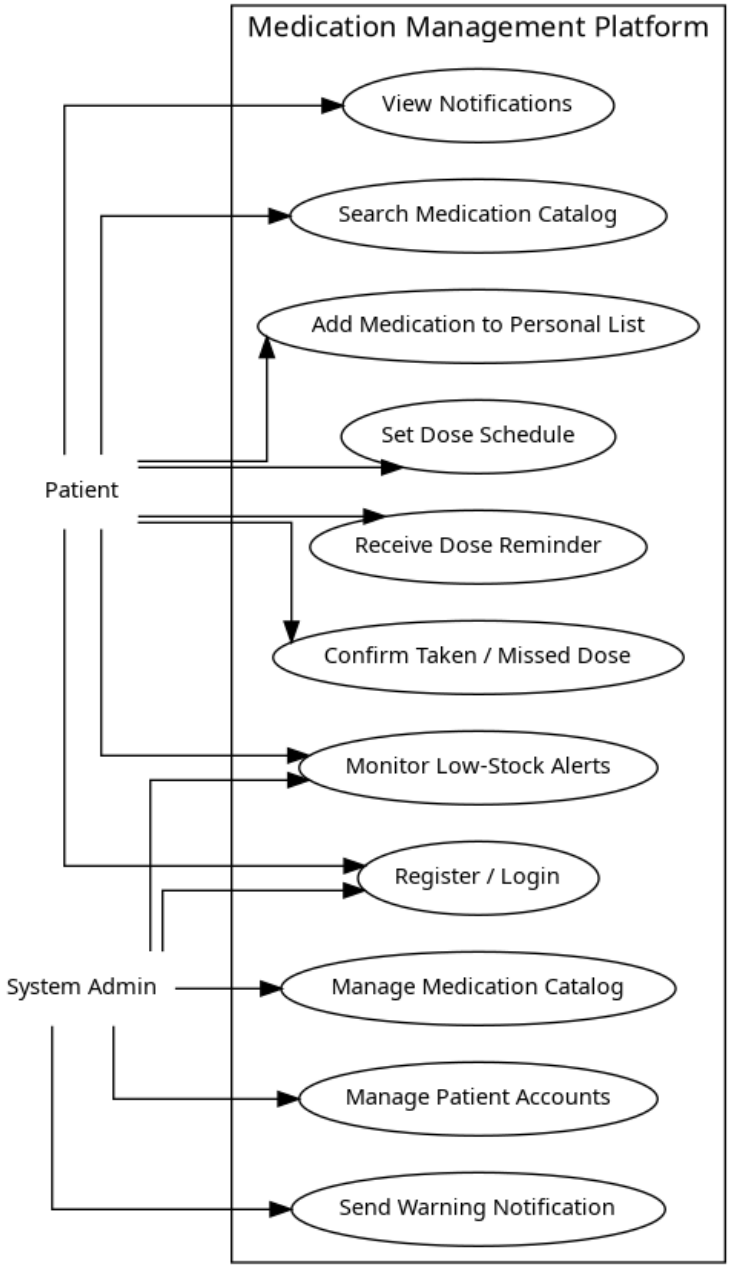

Figure 4-1: Main use case diagram

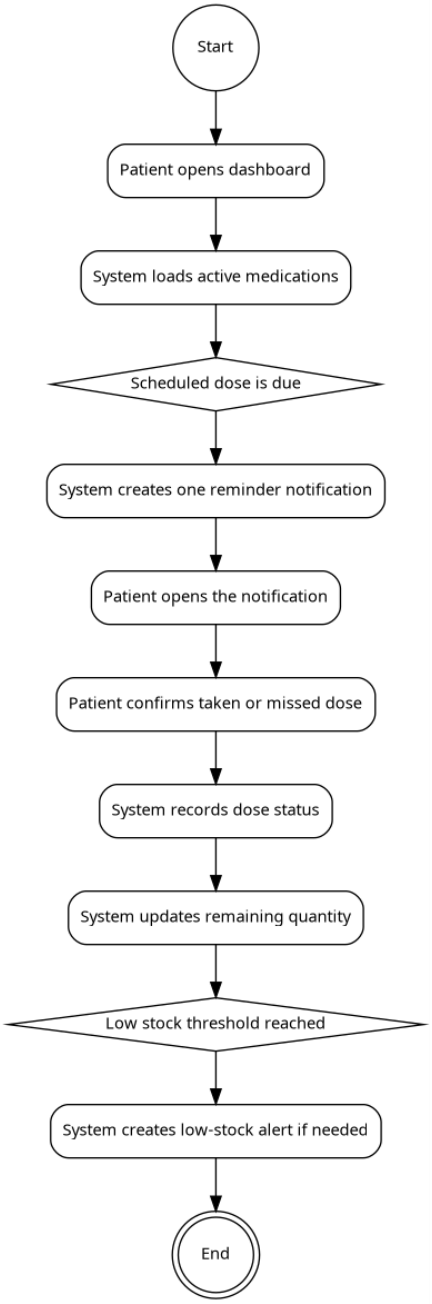

Figure 4-2: Activity diagram for dose reminder and confirmation

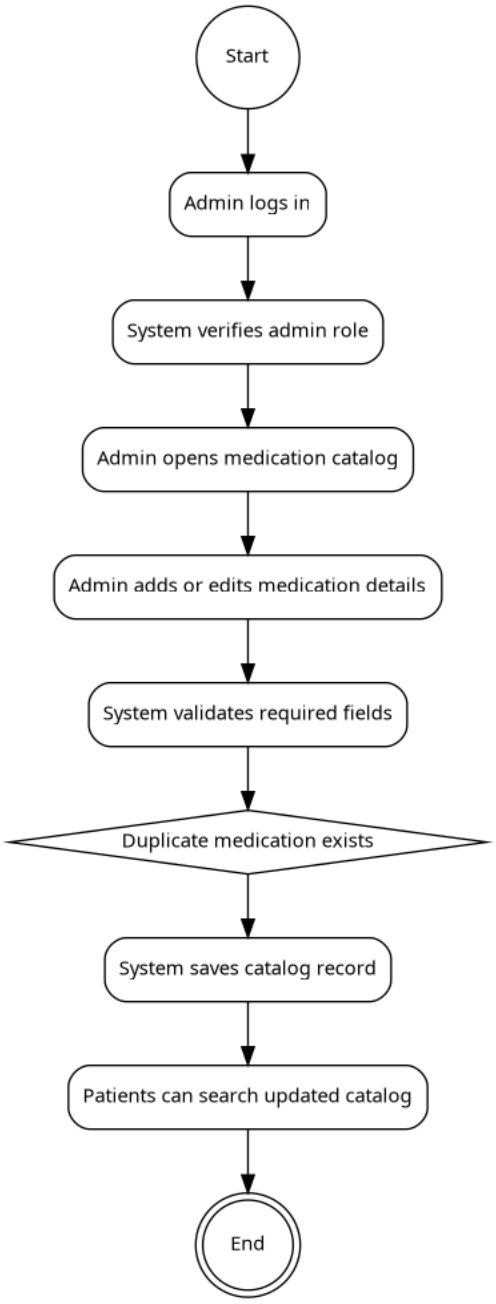

Figure 4-3: Activity diagram for administration of the medication catalog

4.8 Modeling According to Agile Methodology

The Agile model is represented through user stories and a product backlog. Example user stories are: As a patient, I want to receive a reminder when a dose is due so that I do not forget it. As an administrator, I want to add medications to the catalog so that patients can search them. As a patient, I want to receive a low-stock alert so that I can prepare a refill before the medication runs out.

4.9 Requirements Traceability

Table 4-5: Requirements traceability matrix

Requirement	Design / Implementation Artifact	Test Evidence
FR-01	Authentication design, users table, JWT service	Login tests, unauthorized access tests
FR-02	Medication catalog API and search UI	Catalog search tests
FR-03	patient_medications table and add-medication service	Add medication tests
FR-04	dose_schedules table and schedule UI	Schedule creation and edit tests
FR-05	Background scheduler and notification service	Dose reminder creation test
FR-06	Dose record service and notification context	Confirm taken/missed tests
FR-08	Low-stock checker and notification duplicate guard	Low-stock alert test
FR-09	Admin patient management module	Search/edit/disable patient tests
FR-10	Admin catalog management module	Add/update catalog tests
FR-11	Manual warning notification service	Send warning notification test

Chapter 5: System Architecture and Design

5.1 High-Level Design

The platform uses a layered architecture. The user interface communicates with REST endpoints. Controllers validate requests and call business services. Services implement medication, schedule, notification, and administration rules. Repositories or data-access functions communicate with the MySQL database. A background scheduler checks dose times and low-stock thresholds.

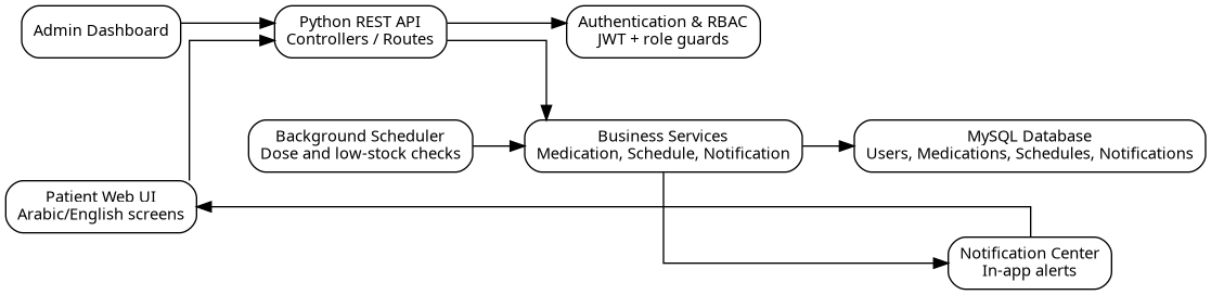

Figure 5-1: System architecture block diagram

5.2 Architectural Pattern

The recommended pattern is a layered web architecture with separation between presentation, API routing, business logic, data access, and persistence. This design is appropriate for a junior project because it is understandable, testable, and extensible. The system can later expose the same backend APIs to a mobile application without redesigning the database.

5.3 Database Design

The relational database is designed to connect users, medications, patient-owned medication records, schedules, dose records, notifications, and revoked tokens. Primary keys identify records, foreign keys maintain relationships, and unique constraints reduce duplicates.

Figure 5-2: Class diagram

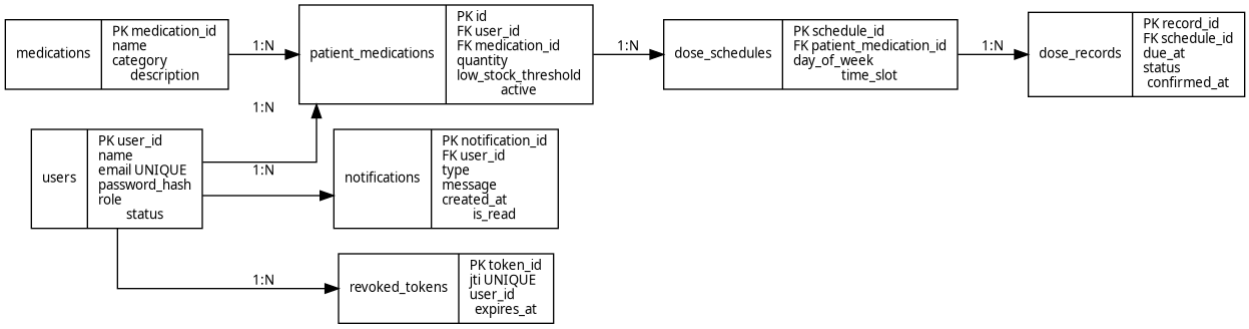

Figure 5-3: Entity relationship diagram

Table 5-1: Relational schema summary

Table	Important Fields	Purpose
users	user_id, name, email, password_hash, role, status	Stores patient and administrator accounts.
medications	medication_id, name, category, description	Stores general catalog information.
patient_medications	id, user_id, medication_id, quantity, low_stock_threshold, active	Stores medications owned by patients.
dose_schedules	schedule_id, patient_medication_id, day_of_week, time_slot	Stores selected dose days and time slots.
dose_records	record_id, schedule_id, due_at, status, confirmed_at	Stores dose events and confirmation status.
notifications	notification_id, user_id, type, message, created_at, is_read	Stores dose, low-stock, and warning notifications.
revoked_tokens	token_id, jti, user_id, expires_at	Stores revoked JWT identifiers for logout/blocklist behavior.

5.4 API Design

Table 5-2: API design summary

Method	Path	Purpose	Input	Output
POST	/api/auth/register	Create a patient account	name, email, password	201 Created or validation error
POST	/api/auth/login	Authenticate a user	email, password	JWT access token and user role
GET	/api/medications	Search catalog	keyword, category	Medication list

Method	Path	Purpose	Input	Output
POST	/api/patient-medications	Add medication to patient list	medication_id, quantity, threshold	Created patient medication record
PUT	/api/schedules/{id}	Update a dose schedule	days, time_slot	Updated schedule
POST	/api/doses/{id}/confirm	Confirm dose status	status: taken/missed	Updated dose record
GET	/api/notifications	View notifications	filters	Notification list
POST	/api/admin/medications	Add catalog medication	name, category, description	Created catalog record
PUT	/api/admin/patients/{id}	Edit patient data	patient fields	Updated patient record
POST	/api/admin/warnings	Send warning notification	recipient(s), message	Created warning notification

5.5 Security and Authorization Design

Security design is based on authentication, authorization, and data ownership. JWT is used for authenticated requests, and role checks separate patient endpoints from administrator endpoints. Patient data must be filtered by the authenticated user identifier to prevent access to other patients' records. Logout is strengthened using a token blocklist or revoked token table.

Chapter 6: Implementation Environment and Tools

6.1 Software Tools and Libraries

Table 6-1: Tools and technologies

Category	Tool / Technology	Purpose
Backend language	Python	Used to implement REST APIs and business logic.
Frontend language	JavaScript	Used to implement interactive browser screens.
Database	MySQL	Used to store users, medication records, schedules, dose records, and notifications.
Editor	Visual Studio Code	Used for coding, extensions, formatting, and debugging.
API testing	Postman or similar tool	Used to test endpoints and request/response behavior.
Version control	Git	Used to track changes and manage branches.
Testing	Manual tests and smoke_test.py	Used to verify patient and administrator journeys.

6.2 Development Environment and Project Structure

The project can be run on a local development machine. A recommended structure separates backend code, frontend code, database scripts, tests, and documentation. This separation improves maintainability and makes it easier to locate bugs.

6.3 Programming Languages and Frameworks

Python is used for backend logic because it is readable, has a large ecosystem, and supports web development. JavaScript is used for the frontend because it runs in the browser and can communicate with REST APIs. The report remains framework-neutral where exact framework details are not specified, but the architecture supports common frameworks such as Flask, Django, or FastAPI on the backend and React or vanilla JavaScript on the frontend.

6.4 Database Server and Operating System

MySQL is used as the database server because it is widely used, well documented, and appropriate for relational data. The platform can run on Windows or Linux during development. For production, a Linux server with environment variables, secure database credentials, and HTTPS is recommended.

6.5 Dependencies and Configuration

Dependencies should be documented in requirements.txt for Python and package.json for JavaScript when applicable. Sensitive configuration such as database password, JWT secret, and environment mode should be stored outside the source code using environment variables or a configuration file that is not committed to version control.

6.6 CI/CD and Testing Environment

A lightweight CI process can run formatting, unit tests, and smoke tests automatically on each commit. In the current academic scope, the minimum testing environment is localhost with a seeded database and repeatable test cases. Future deployment can use Docker and GitHub Actions.

Chapter 7: Software Implementation

7.1 File and Project Structure

A clean structure separates routes, services, repositories, models, validators, scheduler tasks, frontend pages, and tests. The following figure presents a proposed structure that supports maintainability and matches the layered architecture.

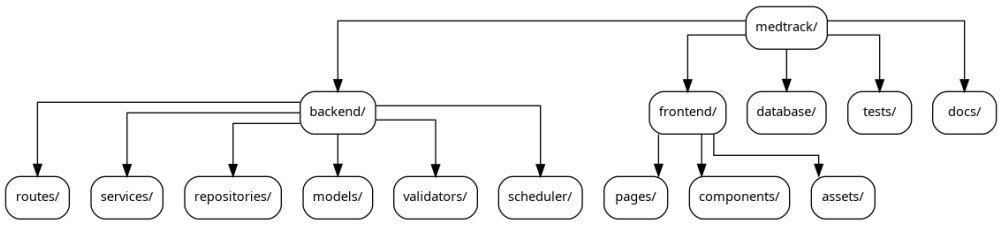

Figure 7-1: Proposed project file structure

7.2 Implementation Modules

Table 7-1: Implementation modules

Module	Description
Authentication module	Register, login, logout, token validation, and role checks.
Medication catalog module	Search, add, update, and browse medication records.
Patient medication module	Add a medication to a patient profile, update quantity, and stop a medication.
Dose schedule module	Create and update dose schedules using selected days and time slots.
Notification module	Create, list, read, and mark notifications as read.
Low-stock module	Check remaining quantity and create alerts without duplicates.
Admin module	Search patients, edit patient data, disable accounts, and send warnings.
Testing module	Smoke tests and manual test records for key journeys.

7.3 API Implementation Example

A representative API flow for confirming a dose is: authenticate the request, verify that the dose record belongs to the current patient, validate the status value, update the dose record, update quantity if applicable, check the low-stock threshold, and return the updated status. This structure prevents random confirmation actions outside the dose context.

7.4 Data Access Layer

The data access layer centralizes database queries and prevents SQL operations from being scattered across the interface or controller code. It should use parameterized queries or an ORM to reduce injection risk. Each repository function should return clear results or raise controlled exceptions.

7.5 Business Logic Layer

The business logic layer contains rules that are specific to the medication domain. Examples include preventing duplicate patient-medication records, preventing duplicate unread low-stock notifications, creating reminders only for active medications, and linking dose confirmation to the correct dose record.

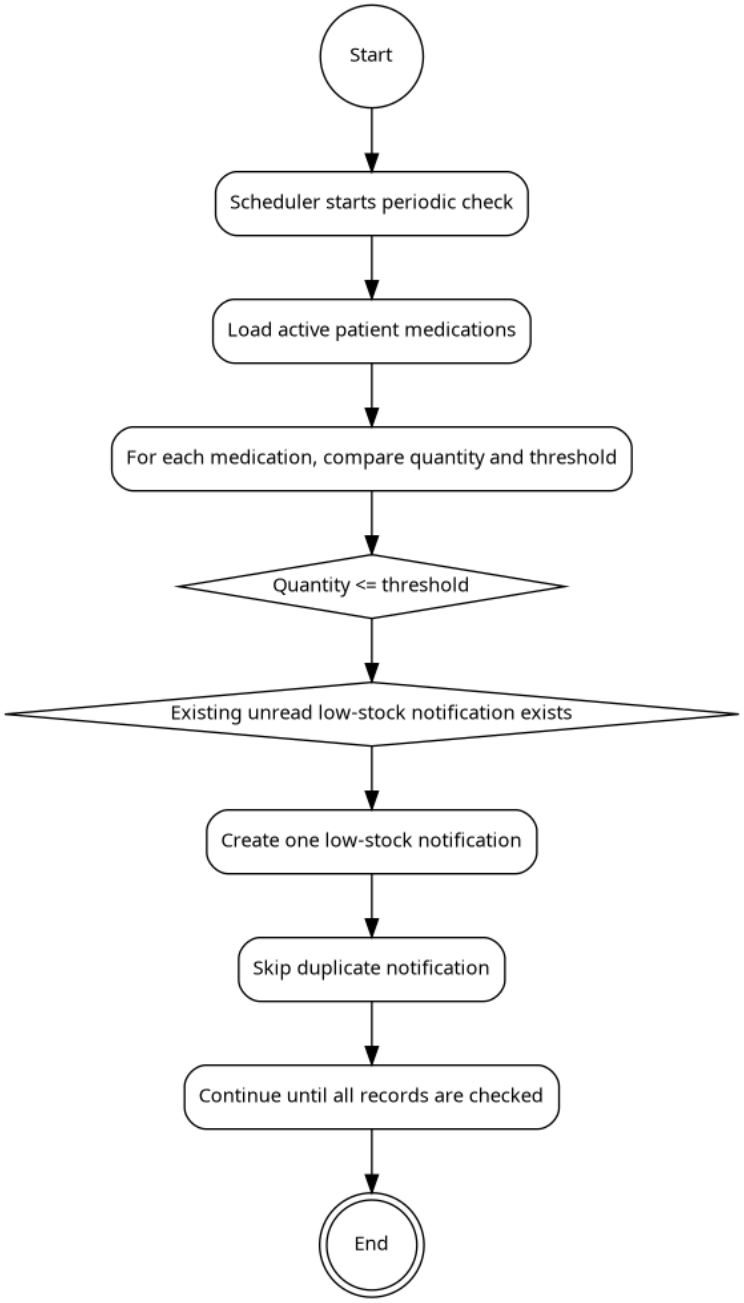

Figure 7-2: Low-stock alert algorithm flow

7.6 User Interface Implementation

The patient interface should contain a dashboard, medication catalog search, personal medication list, schedule editing screen, and notifications screen. The administrator interface should contain patient search, patient details, medication catalog management, and warning notification screens. Buttons should correspond to user goals, and errors should be displayed in clear language.

7.7 Intelligent and Analytics-Oriented Logic

The current version implements rule-based intelligent behavior. It automatically checks dose schedules and stock thresholds, creates contextual notifications, and prevents duplicate alerts. It does not train a machine learning model. Future work can add adherence analytics, missed-dose risk prediction, and dashboards that show adherence rates by time period or medication category.

Chapter 8: Testing and Evaluation

8.1 Test Plan

The test plan verifies whether the system satisfies functional and non-functional requirements. Testing should cover authentication, catalog search, adding medications, schedule editing, reminders, low-stock alerts, notifications, administrator actions, validation errors, and unauthorized access.

8.2 Test Types

- Unit testing for validation functions and business rules.
- Integration testing for API, service, and database interaction.
- System testing for complete patient and administrator journeys.
- Security testing for role guards, token expiration, and ownership checks.
- Usability testing for clarity of screens and error messages.
- Performance testing for response time under local test conditions.

8.3 Test Cases

Table 8-1: Main test cases

ID	Scenario	Input	Expected Result	Status
TC-01	Patient registration	Valid name, email, password	Account is created and can log in	Pass
TC-02	Patient login	Valid credentials	JWT token and patient role returned	Pass
TC-03	Medication search	Keyword or category	Matching medications displayed	Pass
TC-04	Add medication	Medication id, quantity, threshold	Patient medication record created	Pass
TC-05	Edit schedule	Days and time slot	Schedule saved and displayed	Pass
TC-06	Dose reminder	Due scheduled dose	One reminder notification created	Pass
TC-07	Confirm dose	Taken or missed status	Dose record updated and linked to notification	Pass
TC-08	Low-stock check	Quantity below threshold	One low-stock notification created	Pass
TC-09	Duplicate low-stock prevention	Existing unread alert	No duplicate alert created	Pass
TC-	Admin adds	Medication data	Catalog record created	Pass

ID	Scenario	Input	Expected Result	Status
10	catalog item			
TC-11	Admin searches patient	Search term	Matching patient list displayed	Pass
TC-12	Unauthorized access	Patient accesses admin route	Access denied	Pass

8.4 Results and Analysis

Table 8-2: Test results summary

Metric	Value
Total planned test cases	12
Passed test cases	12
Failed test cases	0
High-priority unresolved defects	0
Known limitations	No real pharmacy integration, no mobile push notification, no production load test

8.5 Quality Metrics

Suggested quality metrics include average response time, number of passed test cases, defect density, duplicate-notification rate, and task completion rate in usability testing. In a local academic environment, the most important metric is correct completion of the full medication journey from adding a medication to receiving and confirming reminders.

8.6 Usability Testing

A small usability review can be performed with classmates or potential users. Participants should be asked to log in, search for a medication, add it, create a schedule, and find the notification screen. Observations should focus on clarity of labels, number of steps, and whether users understand the difference between dose reminders and low-stock alerts.

Chapter 9: Results and Analysis

9.1 System Outputs

The main outputs of the system are patient medication records, dose schedules, notification records, low-stock alerts, administrator catalog records, and warning notifications. These outputs demonstrate that the platform connects data management with reminder behavior rather than functioning as a disconnected alarm list.

9.2 Objective Fulfillment

Table 9-1: Objective fulfillment

Objective	Status	Evidence
Manage personal medications	Achieved	Patients can add medications to their own list and view active medications.
Search medication catalog	Achieved	Catalog search and category filtering are part of the proposed API and UI.
Automatic dose reminders	Achieved in rule-based form	The scheduler creates reminders based on schedule records.
Contextual dose confirmation	Achieved	Confirmation is linked to the relevant dose record and notification.
Low-stock alerts	Achieved	Quantity is compared with patient-defined thresholds.
Admin dashboard	Achieved	Admin functions include patient and catalog management and warnings.
Relational database design	Achieved	Tables, keys, and constraints are specified.
AI/analytics features	Partially achieved / future work	The current version has rule-based smart logic; ML analytics are recommended for future work.

9.3 Comparative Analysis with Existing Systems

Compared with generic reminder tools, MedTrack provides stronger data linkage because reminders are associated with patient medication records and dose records. Compared with pharmacy platforms, MedTrack is simpler and more suitable for an academic software engineering project, but it lacks delivery, insurance, and prescription-processing functions.

9.4 Strengths and Limitations

Table 9-2: Strengths and limitations

Type	Description

Type	Description
Strength	Integrated patient medication, schedule, notification, and low-stock data.
Strength	Clear role separation between Patient and Administrator.
Strength	Rule-based prevention of duplicate notifications.
Strength	Database design supports future reports and analytics.
Limitation	No integration with official pharmacy or hospital systems.
Limitation	No machine learning model in the current version.
Limitation	No mobile push notification or SMS integration in the current version.
Limitation	Clinical validation of medication instructions is outside the system scope.

9.5 Key Performance Indicators

Recommended KPIs include percentage of successful dose reminders, duplicate-alert rate, average response time, number of active patient medications, number of confirmed doses, missed-dose rate, and low-stock alert resolution rate. These indicators can form the basis of a future analytics dashboard.

Chapter 10: Conclusion and Recommendations

10.1 Summary of the Project

This report presented a redesigned and English-formatted documentation of the Medication Management Platform. The project addresses medication organization through a web platform that supports patient medication management, dose scheduling, contextual reminders, low-stock alerts, and administrator management. The system was documented according to software engineering standards, including requirements, UML, architecture, database design, implementation environment, testing, and evaluation.

10.2 Challenges

- Defining clear boundaries between a reminder system and a clinical decision-support system.
- Preventing duplicate notifications while maintaining correct reminder behavior.
- Designing a database that supports medication ownership, schedules, dose records, and notifications.
- Balancing simplicity for patients with enough detail for administrators and evaluators.
- Preparing documentation that satisfies academic structure while remaining readable.

10.3 Future Development

- Add mobile applications and push notifications.
- Integrate with pharmacy stock systems or prescription data when legally and technically possible.
- Add adherence analytics dashboards for patients and administrators.
- Train a predictive model to estimate the risk of missed doses based on historical dose records.
- Add multilingual support and accessibility improvements.
- Add exportable reports for medication plans and adherence history.
- Improve security with HTTPS deployment, refresh tokens, audit logs, and stronger password policies.

10.4 Recommendations

Future student teams should define measurable requirements early, connect every requirement to a test case, and avoid adding complex medical features without clinical supervision. Institutions that adopt a similar project should treat it as an organizational and reminder tool rather than a medical prescription system. Any future clinical integration should involve healthcare professionals and comply with privacy and safety requirements.

Appendices

Appendix A: Requirement Collection Summary

Potential requirement collection instruments include short interviews with students or family members who use regular medication, a simple questionnaire about reminder habits, and review of existing reminder applications. Questions should focus on how users remember doses, how they notice low stock, and which notification methods they prefer.

Appendix B: Detailed Design Notes

Detailed design can include expanded class diagrams, component diagrams, data-flow diagrams, state machine diagrams for notification status, API payload examples, and UI mockups. The main report includes the essential diagrams required for evaluation; additional screen captures can be added after final implementation screenshots are available.

Appendix C: Detailed Test Records

Detailed test records should include the tester name, date, environment, input data, expected result, actual result, status, and notes for each test case. Screenshots of fixed defects may be added in this appendix.

Appendix D: User Manual

1. Open the platform URL in a browser.
2. Create a patient account or log in with existing credentials.
3. Search the medication catalog by name or category.
4. Add a medication to the personal list and enter quantity and low-stock threshold.
5. Create a dose schedule by selecting days and time slots.
6. Open the notifications screen when a reminder appears.
7. Confirm the dose as taken or missed from the notification context.
8. For administrator users, log in to the admin dashboard to manage patients, catalog records, and warning notifications.

References

[1] World Health Organization, Adherence to Long-Term Therapies: Evidence for Action. Geneva: World Health Organization, 2003.

[2] R. T. Fielding, Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine, 2000.

[3] OWASP Foundation, OWASP Top 10 Web Application Security Risks, 2021.

[4] K. Schwaber and J. Sutherland, The Scrum Guide: The Definitive Guide to Scrum, 2020.

[5] Oracle, MySQL Reference Manual.

[6] Python Software Foundation, Python Documentation.

[7] Microsoft, Visual Studio Code Documentation.

[8] Original project report: Drug Management Platform, junior project report submitted to the Department of Informatics Engineering.

[9] Graduation Project Thesis Writing Guide, Faculty of Artificial Intelligence Engineering, 2026.